```
            +--------------------+
            |    CSE 421/521     |
            | PROJECT 1: THREADS |
            |   DESIGN DOCUMENT  |
            +--------------------+
```

---- GROUP ----

Qiang Wu <qwu26@buffalo.edu>
Mohammad Sahir <msahir@buffalo.edu>
Siraat Mustafa <siraatmu@buffalo.edu>

---- PRELIMINARIES ----

Alarm Clock, Priority Scheduling, and Advanced Scheduler are implemented.

ALARM CLOCK

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

Added in devices/timer.c:

```
struct sleep_waiter
  {
    uint64_t wake_tick;
    struct semaphore wake;
    struct list_elem elem;
  };

static struct list sleepers;
```

sleep_waiter: A sleep request stored on the calling thread's stack.
wake_tick: The tick when the thread can become ready.
wake: A semaphore that blocks the thread until the timer wakes it.
elem: Connects the request to the sleep list.
sleepers: The pending requests, sorted by wake_tick.

The alarm-clock implementation adds no fields to struct thread.

---- ALGORITHMS ----

>> A2: Briefly describe what happens in a call to timer_sleep(),
>> including the effects of the timer interrupt handler.

timer_sleep() returns immediately if the duration is zero or negative.
Otherwise, it initializes a local semaphore to zero. With interrupts
disabled, it calculates the wakeup tick and inserts the request into
the sorted sleep list. It then restores interrupts and calls sema_down()
to block.

The timer interrupt checks the front of the list. It removes each request whose wakeup tick has arrived and calls sema_up() on its semaphore. It stops at the first request that is not due. This wakes all threads due on the same tick.

>> A3: What steps are taken to minimize the amount of time spent in
>> the timer interrupt handler?

The list is sorted when a thread goes to sleep, so the interrupt handler only visits requests that are due, plus the first one that is not. It does not scan the rest of the list. Inserting a request costs O(n); waking k requests costs O(k + 1).

---- SYNCHRONIZATION ----

>> A4: How are race conditions avoided when multiple threads call
>> timer_sleep() simultaneously?

Each thread has its own request and semaphore. Interrupts are disabled only while reading ticks and inserting into the shared list. On this single-CPU system, another thread cannot run during those operations. The caller restores interrupts before waiting on its semaphore.

>> A5: How are race conditions avoided when a timer interrupt occurs
>> during a call to timer_sleep()?

The request is initialized before it is added to the list. Interrupts are disabled during insertion, so the handler cannot see an unfinished request.

The timer may signal after interrupts are restored but before sema_down(). In that case, sema_up() leaves the semaphore value at one, and sema_down() consumes it without blocking. If sema_down() runs first, the thread blocks until the timer signals it.

The handler removes the request before signaling. Its stack storage stays valid while timer_sleep() is blocked, and it is no longer in the list when the function returns.

---- RATIONALE ----

>> A6: Why did you choose this design? In what ways is it superior to
>> another design you considered?

A semaphore lets each thread block and handles a signal that arrives before the thread starts waiting. Keeping the request on the stack avoids allocating memory or adding fields to struct thread.

An unsorted list would make insertion faster, but the timer would have to scan every request on every tick. Sorting at insertion keeps that work out of the interrupt handler. The cost is O(n) insertion with interrupts disabled, which gets more expensive as the list grows.

PRIORITY SCHEDULING

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

Additions to struct thread in threads/thread.h:

```
int base_priority;
struct lock *waiting_lock;
struct list held_locks;
struct list_elem lock_wait_elem;
```

base_priority: The priority requested by the thread before donation.
waiting_lock: The lock the thread is trying to acquire, or NULL.
held_locks: Locks owned by the thread.
lock_wait_elem: Links a pending acquisition into the lock's contenders.

The existing priority field holds the effective priority. thread.h also
needs a forward declaration of struct lock.

Additions to struct lock in threads/synch.h:

```
struct list contenders;
struct list_elem held_elem;
```

contenders: Threads whose lock acquisitions have not succeeded yet.
held_elem: Links the lock into its holder's held_locks list.

Addition to semaphore_elem in threads/synch.c:

```
struct thread *thread;
```

thread: Identifies the condition waiter, including before it blocks.

The ready list and semaphore wait lists keep their existing list links.

>> B2: Explain the data structure used to track priority donation.
>> Use ASCII art to diagram a nested donation. (Alternately, submit a
>> .png file.)

held_locks groups donations by the lock being requested. Each lock has
a contenders list, and each contender's waiting_lock points to that lock.
Together with lock.holder, these links describe the donation chain.

A contender stays on this list until it acquires the lock. This matters
because sema_up() can make it ready before it has actually taken the lock;
it still needs to donate during that interval.

For H, M, and L with base priorities 50, 30, and 10:

```
    H (base 50)           M (base 30)           L (base 10)
    waits for A -------> holds A
                         waits for B -------> holds B
    effective 50         effective 50         effective 50
```

A holder's effective priority is the maximum of its base priority and the effective priorities of contenders for all its held locks. Donation follows waiting_lock -> holder, stopping after eight holders, a null holder, or a repeated thread.

---- ALGORITHMS ----

>> B3: How do you ensure that the highest priority thread waiting for
>> a lock, semaphore, or condition variable wakes up first?

Ready threads and semaphore waiters are appended at the tail. We scan each list and select the first thread with the highest current effective priority. This keeps equal-priority threads in FIFO order and accounts for donations received while waiting.

sema_up() selects, removes, and unblocks the highest-priority waiter, then increments the semaphore before allowing preemption. Locks use this same semaphore rule. cond_signal() selects through the saved thread pointer. Looking inside its private semaphore is unsafe because the signal can arrive before sema_down(). An early signal stays in the semaphore; a blocked waiter becomes ready. cond_wait() reacquires the monitor lock on return.

A higher-priority ready thread triggers a yield after the caller finishes its state changes. thread_create() performs the same check for a new thread. Idle gives way to any ready thread, including priority zero. thread_unblock() does not yield when called from thread context. Its caller checks whether to yield after completing its state changes. Interrupt handlers request intr_yield_on_return() instead of switching inside the handler.

>> B4: Describe the sequence of events when a call to lock_acquire()
>> causes a priority donation. How is nested donation handled?

If the lock is busy, lock_acquire() sets waiting_lock, registers the caller in contenders, and recomputes the holder's effective priority. If that holder is waiting for another lock, the update continues along the holder chain, up to eight holders. sema_down() then blocks the caller until it can take the semaphore.

After success, the caller removes its contender entry if it registered one, clears waiting_lock, and becomes the holder. The lock is added to held_locks, and the new holder's priority is recomputed from its base priority and contenders for all its held locks. The same ownership updates apply when acquiring an uncontended lock and when lock_try_acquire() succeeds. A failed try registers no wait or
donation. The list and priority changes occur within the synchronization primitives' short atomic sections.

>> B5: Describe the sequence of events when lock_release() is called
>> on a lock that a higher-priority thread is waiting for.

The releasing thread removes the lock from held_locks and clears its holder. It recomputes its effective priority from base_priority and the contenders for its remaining locks. Donations for the released lock are removed; donations for other locks still count.

sema_up() then unblocks the highest-priority waiter and releases the semaphore. The next holder recomputes its priority from the remaining contenders when acquisition succeeds. Once release bookkeeping is complete, the caller yields if a higher-priority thread is ready.

---- SYNCHRONIZATION ----

>> B6: Describe a potential race in thread_set_priority() and explain
>> how your implementation avoids it. Can you use a lock to avoid
>> this race?

Suppose thread_set_priority() reads the donation state and is preempted before writing its result. Another thread adds a higher donation. When the setter resumes, its stale result could overwrite the new priority.

thread_set_priority() disables interrupts, changes base_priority, recomputes the effective priority, and checks whether a higher-priority thread is ready. If so, it yields after the new priority has been stored. The function restores the previous interrupt level before returning. An active donation can keep the effective priority above the new base value. thread_get_priority() returns the effective value.

A sleeping lock would create another scheduling and donation dependency inside the mechanism being updated, and an interrupt handler cannot acquire it. Short interrupt exclusion protects this internal state on Pintos's single CPU. Actual waits still use semaphores and locks.

---- RATIONALE ----

>> B7: Why did you choose this design? In what ways is it superior to
>> another design you considered?

Tracking contenders by lock makes it easy to remove just the donations associated with a released lock. Selecting by current priority also avoids moving a thread whenever a donation changes its position in a ready or waiting queue.

An ordered queue would make selection cheaper, but every priority change would require fixing the thread's position, even when it is blocked on another semaphore. With our unsorted lists, a priority change does not require moving the thread. Selection still takes a linear scan, donation updates scan held locks and contenders, and each thread and lock stores a few extra list links.

ADVANCED SCHEDULER

---- DATA STRUCTURES ----

>> C1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

In threads/fixed-point.h:

```
typedef int32_t fixed_t;
#define FP_SHIFT 14
#define FP_SCALE (1 << FP_SHIFT)
```

fixed_t: A signed 17.14 fixed-point value.
FP_SHIFT: The number of fractional bits.
FP_SCALE: The scale factor for integer and fixed-point conversions.

Additions to struct thread in threads/thread.h:

```
int nice;
fixed_t recent_cpu;
```

nice: The thread's niceness value, inherited from its creator.
recent_cpu: Recent CPU usage, including the fractional part.

In threads/thread.c:

```
static fixed_t load_avg;
static bool tick_in_progress;
static bool ready_changed;
```

load_avg: The moving average of runnable non-idle threads.
tick_in_progress: Marks a timer interrupt whose scheduler updates are not finished.
ready_changed: Records wakeups that need a priority comparison after timer updates.

priority, ready_list, all_list, and thread_mlfqs are reused. Selection
reads current priorities from the single ready list. Fixed-point
multiplication and division use int64_t intermediate values.

The initial thread starts with nice=0 and recent_cpu=0. New ordinary
threads inherit both values before being added to all_list. Idle is
identified before it becomes ready and excluded from CPU and load
accounting. load_avg starts at zero, and idle keeps PRI_MIN priority.
MLFQS ignores manually supplied priorities and bypasses donation.
Changing nice recalculates priority and checks preemption. Nice values
range from -20 to 20; thread_get_nice() returns the stored value.

---- ALGORITHMS ----

>> C2: Suppose threads A, B, and C have nice values 0, 1, and 2. Each
>> has a recent_cpu value of 0. Fill in the table below showing the
>> scheduling decision and the priority and recent_cpu values for each
>> thread after each given number of timer ticks:

timer	recent_cpu			priority			thread
ticks	A	B	C	A	B	C	to run
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A
24	16	8	0	59	59	59	C
28	16	8	4	59	59	58	B
32	16	12	4	59	58	58	A
36	20	12	4	58	58	58	C

>> C3: Did any ambiguities in the scheduler specification make values
>> in the table uncertain? If so, what rule did you use to resolve
>> them? Does this match the behavior of your scheduler?

A, B, and C start ready in that order and remain runnable. Each row
shows values after the listed tick's accounting and priority update.
The final column gives the thread selected for the next four-tick slice.

At slice expiry, the running thread joins the ready-list tail. The
scheduler scans the list and selects the first thread with the highest
priority. Thus B precedes A at tick 8, and C precedes both at tick 24.
A is selected again at tick 4 because its priority is still higher.
With TIMER_FREQ=100, no one-second update occurs in this table.

>> C4: How is the way you divided the cost of scheduling between code
>> inside and outside interrupt context likely to affect performance?

Every tick adds one recent_cpu unit to the running non-idle thread,
then wakes due sleepers. Every TIMER_FREQ ticks, load_avg is updated
before recent_cpu is recomputed for all non-idle threads, including
blocked threads. Every fourth global tick updates their priorities.
When both boundaries coincide, the priority formula uses the new values.

```
priority = PRI_MAX - recent_cpu / 4 - 2 * nice
load_avg = (59/60) * load_avg + (1/60) * ready_threads
recent_cpu = (2*load_avg)/(2*load_avg+1) * recent_cpu + nice
```

ready_threads counts the ready list plus the running non-idle thread,
after alarm wakeups. thread_tick() records CPU use and the time slice.
thread_tick_end() runs after the alarm wake loop and performs the periodic
updates. If priorities changed or a sleeper woke, it compares the current
and ready priorities and requests intr_yield_on_return() when needed.
Time-slice expiration also requests a switch. All updates finish before
interrupt return; wakeups in the timer handler do not schedule using
partially updated priorities.

Periodic updates cost O(n) for n threads. Dispatch costs O(r) for r ready threads, and waking k sleepers adds O(k+1). All-thread scans occur only at the specified boundaries. Thread creation and thread_set_nice() calculate one thread's priority outside interrupt context, with short atomic sections for shared state. With more threads, the O(n) updates increase time spent handling timer interrupts.

---- RATIONALE ----

>> C5: Briefly critique your design, pointing out advantages and
>> disadvantages in your design choices. If you were to have extra
>> time to work on this part of the project, how might you choose to
>> refine or improve your design?

One ready list keeps selection simple and works with both schedulers. Reading priorities at dispatch avoids an ordering that becomes stale after a periodic update. Blocked threads receive updated values without being moved between queues.

The main costs are O(r) dispatch and O(n) periodic scans. With more time, we would consider 64 FIFO ready queues to reduce dispatch cost. That would require moving ready threads after priority changes and preserving the same tie order.

>> C6: The assignment explains arithmetic for fixed-point math in
>> detail, but it leaves it open to you to implement it. Why did you
>> decide to implement it the way you did? If you created an
>> abstraction layer for fixed-point math, that is, an abstract data
>> type and/or a set of functions or macros to manipulate fixed-point
>> numbers, why did you do so? If not, why not?

The inline helpers in threads/fixed-point.h operate on signed 17.14 values, with F = 1 << 14. They keep the scale conversions in one place and make the scheduler formulas easier to check.

An integer n becomes n * F, and adding it to x uses x + n * F. Two fixed-point values multiply as ((int64_t) x * y) / F and divide as ((int64_t) x * F) / y. Casting to int64_t avoids overflow in a 32-bit intermediate product; the final scaled result must still fit in fixed_t. Multiplication and division by F also handle negative operands without relying on signed shifts.

Stored values keep their fractions, including negative recent_cpu. The priority formula uses the combined 64-bit numerator ((PRI_MAX - 2 * nice) * FP_SCALE * 4 - recent_cpu), divided by 4 * FP_SCALE once, then clamped to 0..63. This keeps all fractional bits until the final truncation. For a valid positive priority, this rounds down. load_avg similarly combines its weighted numerator before division by 60. The recent_cpu coefficient is calculated before it is multiplied by the stored CPU value.

The load and recent CPU getters multiply by 100 in a wide intermediate and round to nearest. For scaled x, use (x + F/2)/F when x is nonnegative and (x - F/2)/F when negative, with division toward zero. Separate helpers for truncation and rounding make that distinction explicit.